正则表达式

1.1 什么是正则表达式

正则表达式，又称规则表达式，（Regular Expression，在代码中常简写为regex、regexp或RE），是一种文本模式，包括普通字符（例如，a 到 z 之间的字母）和特殊字符（称为"元字符"）。正则表达式使用单个字符串来描述、匹配一系列符合某个句法规则的字符串，通常被用来检索、替换那些符合某个模式（规则）的文本。通俗的说，正则表达式就是一种语法规则，用来匹配文本中的文本。

正则表达式非常强大，不仅在UNIX系统中应用广泛，近二十年来，在Windows系统中也得到极大的发展，现如今主流的操作系统包括Linux、Unix、Windows和主流的开发语言包括C/C++、Java、JavaScript、C#、Python、Go、PHP等。

一个正则表达式的匹配工具：

https://regex101.com/

1.2 正则表达式的语法规则

1.2.1 匹配什么字符

1. **.**：匹配任意的字符（换行符除外）
2. **\d**：匹配数字字符
3. **\D**：匹配非数字字符
4. **\w**：匹配单词字符（英文、数字、下划线）
5. **\W**：匹配非单词字符
6. **\s**：匹配空白符（包括换行符、Tab键、空格）
7. **\S**：匹配非空白字符

1.2.2 匹配多少次

1. *****: 出现0次或多次（贪婪）
2. **+**：出现1次或多次（贪婪）
3. **?** ：出现0次或1次（懒惰）
4. **{n}**：出现n次
5. **{n, m}**：出现n~m次

华清远见|元宇宙实验中心 yyzlab.com.cn

6. **{n，}**：出现n次以上

1.2.3 在哪里匹配

1. **^**：在行首的位置匹配
2. **$**：在行尾的位置匹配
3. **\b**：表示匹配单词边界。（比如\bword，可以匹配word、words，但不会匹配sword）

1.2.4 匹配指定格式的字符

1. 使用**()**匹配指定格式的字符，比如：

```
(ab)：表示在文本中只匹配ab这两个字符，且必须相邻
(a|b)：表示在文本中匹配a或者b这两个字符，不一定相邻。
```

注意：（）会把匹配到的内容保存在内存上，开发者可以使用\n来代表一个匹配模式中第n个()中匹配到的内容。

2. 使用**[]**匹配指定类别的字符串，比如：

```
[abcd]：表示匹配a或匹配b或匹配c或匹配d
[a-d]：表示匹配a或匹配b或匹配c或匹配d
[a-zA-Z0-9]：表示匹配所有的大小写英文和数字
[^0-9]：表示匹配除了数字之外的所有字符
```

1.2.5 单行模式与多行模式

1. **单行模式**：在单行模式下，**.**可以匹配任何的字符，包括换行符，并且整个文本会被认为是一个完整的文本，使用**^**和**$**只能匹配到文本的开头和结尾。
2. **多行模式**：在多行模式下，**.**就不可以匹配换行符了，使用**^**和**$**可以匹配到每一行的开始和结束。

1.2.6 贪婪匹配与懒惰匹配

在正则表达式中，**贪婪匹配**和**懒惰匹配**是两种不同的数量匹配策略。它们决定了量词（比如*****、**+**、**?** 、{n，m}等）如何匹配尽可能多的或尽可能少的字符。

1. **贪婪模式**：指在正则表达式中的量词会尽可能多地匹配字符。
2. **懒惰模式**：指在正则表达式中的量词会尽可能少地匹配字符。

华清远见 HQYJ.COM
华清远见云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

在Python的`re`模块中，你可以通过在量词后面添加一个问号（?）来设置非贪婪（懒惰）模式。相反，如果不加问号，则默认为贪婪模式。

1.2.7 ？的几种用法

1. ？作为限定符，表示其修饰对象只能出现0次或1次。
2. ？放在量词前，表示将匹配模式改为懒惰匹配模式。
3. （?=pattern）：表示匹配位置后面必须跟着pattern模式的字符，匹配结果并不包括这个模式的字符串。
4. （?!pattern）：表示匹配位置后面不能跟着pattern模式的字符，匹配结果并不包括这个模式的字符串。
5. （?<=pattern）：表示匹配位置前面必须跟着pattern模式的字符，匹配结果并不包括这个模式的字符串。
6. （?<!pattern）：表示匹配位置前面不能跟着pattern模式的字符，匹配结果并不包括这个模式的字符串。
7. （?:pattern）：表示将 pattern 包含在一个分组中，但不把这个分组的匹配结果保存到分组编号中。

1.3 Python中的re库

在Python中，关于正则表达式的功能都在re库中，下面是re库中的一些常用的函数及其功能：

1.3.1 re.match

该函数尝试从字符串的起始位置匹配，如果起始位置没有匹配成功的话，就返回None，否则会返回一个匹配对象，匹配对象的方法可参考下面的**附录2表格。**

函数原型：**re.match(pattern，string，flags=0)**

pattern：正则表达式的格式。

string：被匹配的文本。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

常用的应用场景有：

1. **验证输入格式**：检查用户输入是否符合特定的格式，例如电子邮件、电话号码、日期等。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

2. **提取信息**：从字符串的开始位置提取符合正则表达式模式的字串，如提取文件名等。
3. **数据解析**：在处理日志文件或配置文件时，可以用来解析每行的开始部分，获取关键信息。

```
import re
while True:
    email = input('请输入有效的邮箱：')
    pattern = r'[0-9a-zA-Z+-_.]+@[0-9a-zA-Z.-]+\.[a-zA-Z]{2,}'
    res = re.match(pattern, email)
    if res:
        print('您的邮箱格式正确')
        break
    else:
        print('您的邮箱格式有误，请重新输入')
```

1.3.2 re.search

该函数从文本中获取第一个符合正则表达式模式的字符的位置，并返回一个匹配对象，如果没有匹配到，就返回None。

函数原型：**re.search(pattern，string，flags=0)**

pattern：正则表达式的格式。

string：被匹配的文本。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

常用的应用场景有：

1. **检查字符串是否包含子串**：检查一个字符串是否包含某个特定的文本。
2. **提取数据**：从字符串中的任意位置提取信息，例如从一个文本段落中提取所有提到的日期。
3. **搜索文件**：在某文件中查找特定的消息或事件。

```
import re

text = '''
gjiaokremgn
vkn24124pa2
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
13315561889
hbvuijrbnio
15633567985
'''

pattern = r'^[1]{1}[3589]{1}[0-9]{9}$'
res = re.search(pattern, text, re.M)
if res:
    print(res.group())
    print(res.start())
```

1.3.3 re.findall

该函数从文本中寻找所有与模式匹配的子串，并将所有的匹配结果存储到一个列表中进行返回，如果没有匹配成功会返回一个空列表。

函数原型：**re.findall(pattern，string，flags=0)**

pattern：正则表达式的格式。

string：被匹配的文本。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

常用的应用场景有：

1. **提取多个子串**：当你需要在字符串中找到所有匹配特定模式的字串时。
2. **文本分析**：在文本中，提取文本中特定的词汇、短语或模式。

```
import re

text = '''
gjiaokremgn
vkn24124pa2
13315561889
hbvuijrbnio
15633567985
'''

pattern = r'^[1]{1}[3589]{1}[0-9]{9}$'
res = re.findall(pattern, text, re.M)
print(res)
```

华清远见|云宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

1.3.4 re.sub

该函数的作用就是将文本中与模式匹配的部分替换为其他的内容。

函数原型：**re.sub(pattern，repl，string，count，flags=0)**

pattern：正则表达式的格式。

repl：这是替换文本或一个函数。如果是文本，就是将匹配到的内容替换为该文本；如果是函数，会在函数中进行文本处理的操作。

string：被匹配的文本。

count（可选）： 这是可选参数，表示替换的最大次数。默认值为 0，表示替换所有匹配项。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

常用的应用场景有：

1. **文本格式化**：将文本中的特定模式的文本替换为另一种格式。
2. **数据清洗**：在处理数据时，移除或替换无效或不需要的字符，比如在一系列文本中删除非数字字符以清理电话号码。
3. **敏感信息脱敏**：在显示或存储数据前，将敏感信息（比如身份证号、手机号）的部分内容替换为星号。

```
import re

text = "1999.12.31是20世纪的最后一天"
pattern = r"(\d{4})\.(\d{2})\.(\d{2})"
replacement = r"\1年\2月\3日"

new_text = re.sub(pattern, replacement, text)
print(new_text)
```

```
import re

# 原始字符串
text = "Hello 123 World 456"

# 定义一个正则表达式模式，匹配数字
pattern = r'\d+'
```

华清远见嵌入式实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
# 定义一个替换函数
def replace(match):
    # 将匹配到的数字乘以2
    number = int(match.group())
    return str(number * 2)

# 使用 re.sub 和替换函数
new_text = re.sub(pattern, replace, text)

print(new_text)  # 输出 "Hello 246 World 91
```

```python
import re


def desensitize_info(text):
    # 身份证号的正则表达式
    id_card_pattern = r'(\d{6})(\d{4})(\d{4})(\d{3})(\d|X)'
    # 手机号的正则表达式
    phone_pattern = r'(\d{3})(\d{4})(\d{4})'

    # 脱敏身份证号
    text = re.sub(id_card_pattern, r'\1********\4\5', text)
    # 脱敏手机号
    text = re.sub(phone_pattern, r'\1****\3', text)

    return text


# 示例文本
text = "我的身份证号是12345619870605123X，手机号是13812345678"

# 脱敏处理
desensitized_text = desensitize_info(text)

print(desensitized_text)
```

华清远见元宇宙实验中心 yyzlab.com.cn

1.3.5 re.split

该函数的作用就是将某文本根据匹配模式进行分割，并将分割后的结果放入列表中返回。

函数原型：**re.split(pattern，string，maxsplit，flags=0)**

pattern：正则表达式的格式。

string：被匹配的文本。

maxsplit（可选）：这是可选参数，表示最大分割次数。默认值为 0，表示分割所有匹配项。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

```
import re

text = "apple, banana, orange, watermelon"
fruits = re.split(r',\s*', text)
print(fruits)
```

1.3.6 re.compile

该函数会预先编译正则表达式要匹配的模式，并会返回一个正则表达式的对象，该对象与re.match返回的对象不同，该对象可以调用上面的函数。

函数原型：**re.compile(pattern, flags=0)**

pattern：要匹配的正则表达式。

flags（可选）：标志位，用于控制正则表达式的匹配方式，如：是否区分大小写、设置多行匹配模式等，具体有哪些标志可参考下面的**附录1表格。**

常见的应用场景：

多次匹配：当你需要在一个较长的文本中多次应用同一个正则表达式时，使用**re.compile**可以避免每次匹配时都重新编译表达式。

```
import re

# 编译一个正则表达式模式，用于匹配手机号
email_pattern = re.compile(r'[1]{1}[3589]{1}[0-9]{9}')
```

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```
# 使用编译后的模式进行多次匹配
text = '''
1o23j0 rde0
13309876543
cvbnoikdjgv
10248895490
-/*-+68-*8+
'''
emails = email_pattern.search(text)

print(emails)
```

附录1：flags参数

flags	说明
re.IGNORECASE 或 re.I	使匹配对大小写不敏感
re.LOCALE 或 re.L	做本地化识别的匹配，会影响\w、\W、\b 和 \B
re.MULTILINE 或 re.M	多行匹配，使 ^ 和 $ 能够匹配每一行的行首和行尾
re.DOTALL 或 re.S	单行匹配，使.特殊字符能够匹配任何字符，包括换行符
re.VERBOSE 或 re.X	允许你编写更易于阅读的正则表达式，忽略空白字符和注释
re.ASCII 或 re.A	使得 \w、\W、\b、\B 和大小写字母的匹配只基于 ASCII 字符集，而不是 Unicode

附录2：匹配对象的方法

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

方法	说明
group()	返回被匹配的字符串
start()	返回匹配开始的位置
end()	返回匹配结束的位置
span()	返回一个元组包含开始和结束的位置

华清远见|元宇宙实验中心
yyzlab.com.cn